

Google高级软件工程师Brett Slatkin融合自己多年Python开发实战经验，深入探讨编写高质量Python代码的技巧、禁忌和最佳实践

涵盖Python 3.x和Python 2.x主要应用领域，汇聚59条优秀实践原则、开发技巧和便捷方案，包含大量实用范例代码

Effective Python

59 Specific Ways to Write Better Python

Effective Python

编写高质量Python代码的
59个有效方法

[美] 布雷特·斯拉特金 (Brett Slatkin) 著
爱飞翔 译



机械工业出版社
China Machine Press

有关此电子书的说明

本人可以帮助你找到你要的PDF电子书，计算机类，文学，艺术，设计，医学，理学，经济，金融等等。质量都很清晰，为方便读者阅读观看，每本100%都带可跳转的书签索引和目录，只要您提供给我书的相关信息，一般我都能找到，如果您有需求，请联系我 QQ1779903665。

PDF代找说明：

本人已经帮助了上万人找到了他们需要的PDF，其实网上有很多PDF,大家如果在网上不到的话，可以联系我QQ，大部分我都可以找到，而且每本100%带书签索引目录。因PDF电子书都有版权，请不要随意传播，如果您有经济购买能力，请尽量购买正版。

提供各种书籍的pd电子版代找服务，如果你找不到自己想要的书的pdf电子版，我们可以帮您找到，如有需要，请联系 QQ 1779903665.

备用:QQ 461573687

声明：本人只提供代找服务，每本100%索引书签和目录，因寻找和后期制作pdf电子书有一定难度，仅收取代找费用。如因PDF产生的版权纠纷，与本人无关，我们仅仅只是帮助你寻找到你要的pdf而已。

Effective Python

59 Specific Ways to Write Better Python

Effective Python

编写高质量Python代码的
59个有效方法

[美] 布雷特·斯拉特金 (Brett Slatkin) 著

爱飞翔 译



机械工业出版社
China Machine Press

图书在版编目 (CIP) 数据

Effective Python: 编写高质量 Python 代码的 59 个有效方法 / (美) 斯拉特金 (Slatkin, B.) 著; 爱飞翔译. —北京: 机械工业出版社, 2016.1

(Effective 系列丛书)

书名原文: Effective Python: 59 Specific Ways to Write Better Python

ISBN 978-7-111-52355-0

I. E… II. ①斯… ②爱… III. 软件工具—程序设计 IV. TP311.56

中国版本图书馆 CIP 数据核字 (2015) 第 305873 号

本书版权登记号: 图字: 01-2015-2812

Authorized translation from the English language edition, entitled *Effective Python: 59 Specific Ways to Write Better Python*, 9780134034287 by Slatkin, published by Pearson Education, Inc., Copyright © 2015.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

Chinese simplified language edition published by Pearson Education Asia Ltd., and China Machine Press Copyright © 2016.

本书中文简体字版由 Pearson Education (培生教育出版集团) 授权机械工业出版社在中华人民共和国境内 (不包括中国台湾地区和香港、澳门特别行政区) 独家出版发行。未经出版者书面许可, 不得以任何方式抄袭、复制或节录本书中的任何部分。

本书封底贴有 Pearson Education (培生教育出版集团) 激光防伪标签, 无标签者不得销售。

Effective Python

编写高质量 Python 代码的 59 个有效方法

出版发行: 机械工业出版社 (北京市西城区百万庄大街 22 号 邮政编码: 100037)

责任编辑: 关 敏

责任校对: 董纪丽

印 刷: 三河市宏图印务有限公司

版 次: 2016 年 1 月第 1 版第 1 次印刷

开 本: 186mm × 240mm 1/16

印 张: 14

书 号: ISBN 978-7-111-52355-0

定 价: 59.00 元

凡购本书, 如有缺页、倒页、脱页, 由本社发行部调换

客服热线: (010) 88379426 88361066

投稿热线: (010) 88379604

购书热线: (010) 68326294 88379649 68995259

读者信箱: hzit@hzbook.com

版权所有 • 侵权必究

封底无防伪标均为盗版

本书法律顾问: 北京大成律师事务所 韩光 / 邹晓东

Praise 本书赞誉

“Slatkin 所写的这本书，其每个条目（item）都是一项独立的教程，并包含它自己的源代码。这种编排方式使我们可以随意跳读：大家可以按照学习的需要来浏览这些条目。本书涉及的话题十分广泛，作者针对这些话题，给出了相当精练而又符合主流观点的建议，我把这本书推荐给中级 Python 程序员。”

——Brandon Rhodes, Dropbox 的软件工程师、2016 ~ 2017 年 PyCon 会议的主席

“我用 Python 写了很多年程序，自认为对 python 已经了解得很透彻了。但在看过这本讲解决窍和技巧的好书之后我才发现，其实我还能把 Python 代码写得更高效（例如，使用内置的数据结构）、更易读（例如，设定只允许通过关键字形式来指定的参数），以令其更加符合 Python 风格（例如，用 zip 函数来同时迭代两个列表）。”

——Pamela Fox, Khan 学院的教师

“当初我刚从 Java 转向 Python 时，要是能先看到这本书的话，那就能节省好几个月的时间。这本书使我意识到：以前反复编写的那些代码，都不是很符合 Python 的编程风格。这本书包含了 Python 语言的绝大部分必备知识，使我们无需通过数月乃至数年的艰难探索，即可逐个了解它们。本书的内容非常丰富，从 PEP8 的重要性和 Python 语言的主要编程习惯开始，然后谈到如何设计函数、方法和类，如何高效地使用标准库，以及如何设计高质量的 API，最后，又讲了测试及性能问题。新手和老手都可以通过这本优秀教程来领略 Python 编程的真谛。”

——Mike Bayer, SQLAlchemy 的创立者

“这本书会清楚地告诉你如何改善 Python 代码的风格及函数的质量，它会令你的 Python 技能更上一层楼。”

——Leah Culver, Dropbox 的开发者代言人 (developer advocate)

“这是一本极好的书，对其他编程语言较有经验的开发者，可以通过本书迅速学习 Python，并了解更符合 Python 风格的基础语言结构。本书内容清晰、简明，而且易于理解，只需阅读某个条目或某一章，即可单独研究某个话题。书中讲解了大量纯 Python 的语言结构，使读者不会把它们与 Python 生态圈中的其他复杂事物相混淆。经验更多的开发者可以通过书中提供的一些深度范例来了解自己尚未遇到的语言特性，以及原来不常使用的语言功能。作者肯定是一位非常熟悉 Python 的人，他用自己丰富的经验来给读者指出各种经常出现的 bug 以及经常出错的写法。另外，本书也恰当地说明了 Python 2.X 与 Python 3.X 之间的微妙区别，大家在各种版本的 Python 之间迁移时，可以把本书用作参考资料。”

——Katherine Scott, Tempo Automation 的软件主管

“这是一本对初级开发者和熟练开发者都适用的好书。代码范例及其讲解都写得非常细致、非常简洁、非常透彻。”

——C. Titus Brown, 加州大学戴维斯分校副教授

“这本参考书非常有用，它提供了很多高级的 Python 用法，并讲解了如何构建更清晰、更易维护的软件。把书中的建议付诸实践，就可以令自己的 Python 技能得到提升。”

——Wes McKinney, pandas 程序库的创立者《Python for Data Analysis》^①

的作者、Cloudera 的软件工程师

① 中文版为《利用 Python 进行数据分析》，已由机械工业出版社引进出版。——编辑注

The Translator's Words 译者序

自 Scott Meyers 撰写的《Effective C++》问世以来，出现了很多以 Effective 命名的技术书籍。Effective 一词，并不单单局限于执行速度层面的高效率，同时有着令代码易于阅读、易于测试且易于维护等意思，此外，它还蕴涵着易于扩展、易于修改和易于多人协作等更为高阶的理念。

因此，从上述宏观层面来看，Effective 式的心得手册，无论是对初学者还是熟练者，都有较大意义。本书自然也不例外。对于初学者来说，书中展示了该语言的大体轮廓，使我们能够知道 Python 的强项和弱项。在知道了这些特性之后，开发者就可以结合自己的兴趣与需求，有选择、有顺序地学习。

而对于熟练者来说，则可以把书中的心得与自己的经验相比对，看看自己还有哪些区域尚未深入研究，同时思考一下书中的方案与自己常用的方案各有什么优点与缺点。

从本书各条技巧的具体编排方式来看，本书既可以像字典那样查阅，也可以像普通图书那样通读。很多条目都是用渐进的方式来编写的。作者不会在一开始就给出最佳方案，而是会先从简单的写法入手，逐步发现其缺点并加以完善，最后总结出一套便于使用且易于扩充的解决办法。

这样的演进方式既适用于本书所列的各个场景，也适用于日常的编程工作。如果能通过这些具体的条目来培养一套分析并解决问题的思路，那就可以更加深刻地体会 Python 语言的设计哲学及实践艺术。很多 Python 开发者都崇尚 Pythonic 编程方式，这种 Pythonic 方式不仅应该体现在代码风格和项目规范之中，而且更应该体现在思维模式和架构设计层面，这一点，我想应该是 Effective 系列的书籍值得反复品味的缘由吧。

虽说这 59 条技巧并不能涵盖所有的 Python 领域，但在经常接触的那几个主要领域中，它们却是相当有代表性和启发性的。我们可以把这些技巧以自己的方式实现出

来，并封装成模块及软件包，以便在后续的工作中使用。而对于本书没有专门涉及的领域，如游戏开发、图形绘制、网络通信等，大家不妨也沿用 Effective 书系的一贯做法，把自己的经验总结成条目，进而以博客或开源项目的形式互相交流。这可以说是对 Effective 理念的一种延伸和发展。

由于许多 Python 开发者都同时具备 C++ 及 Java 等其他语言的开发背景，所以 Python 中的很多概念都有好几种不同的称呼方式，而这些术语的中文翻译，自然也就呈现出了一词多译的现象。本书将尽量采用较为折中的办法来处理这些问题。

本书的翻译过程中，得到了机械工业出版社华章公司诸位编辑和工作人员的帮助，在此深表谢意。

由于译者水平有限，不足与疏漏之处，请大家发邮件至 castarstormlee@gmail.com，或访问 github.com/jeffreybaoshenlee/zh-translation-errata-effective-python/issues 留言，给我以批评和指教。该网页还有《中英文词汇对照表》，以供参考。

Preface 前言

Python 编程语言很强大、很有魅力，但同时也很独特，所以掌握起来比较困难。许多程序员从他们所熟悉的语言转入 Python 之后，没能把思路打开，以致写出的代码无法完全发挥出 Python 的特性，而另外一些程序员则相反，他们滥用 Python 的特性，导致程序可能在将来出现严重问题。

本书会深入讲解如何以符合 Python 风格的（Pythonic）方式来编写程序，这种方式就是运用 Python 语言的最佳方式。笔者假定你对这门语言已经有了初步了解。编程新手可以通过本书学到各种 Python 功能的最佳用法，而编程老手则能够学会如何自信地运用一种功能强大的新工具。

笔者的目标是令大家学会用 Python 来开发优秀的软件。

本书涵盖的内容

本书每一章都包含许多互相关联的条目，大家可以按照自己的需要，随意阅读这些条目。每个条目都包含简明而具体的教程，告诉你应该如何更高效地编写 Python 程序。笔者在每个条目里面都给出了建议，告诉大家应该怎样做、应该避免哪些用法，以及如何在各种做法之间求得平衡，并解释了笔者所选的做法好在哪里。

本书中的各项条目，适用于 Python 3 和 Python 2（请参阅本书第 1 条）。对于 Jython、IronPython 或 PyPy 等其他运行时环境，大部分条目应该同样适用。

第 1 章：用 Pythonic 方式来思考

Python 开发者用 Pythonic 这个形容词来描述具有特定风格的代码。这种风格是大家在使用 Python 语言进行编程并相互协作的过程中逐渐形成的习惯。本章讲解如何以

该风格来完成常见的 Python 编程工作。

第 2 章：函数

Python 中的函数具备多种特性，这可以简化编程工作。Python 函数的某些性质与其他编程语言中的函数相似，但也有些性质是 Python 独有的。本章介绍如何用函数来表达意图、提升可复用程度，并减少 bug。

第 3 章：类与继承

Python 是面向对象的语言。用 Python 编程时，通常需要编写新类，并定义这些类应该如何通过其接口及继承体系与外界相交互。本章讲解如何使用类和继承来表达对象所应具备的行为。

第 4 章：元类及属性

元类（metaclass）及动态属性（dynamic attribute）都是很强大的 Python 特性，然而它们也可能导致极其古怪、极其突然的行为。本章讲解这些机制的常见用法，以确保读者写出来的代码符合最小惊讶原则（rule of least surprise）。

第 5 章：并发及并行

用 Python 很容易就能写出并发程序，这种程序可以在同一时间做许多件不同的事情。我们也可以通过系统调用、子进程（subprocess）及 C 语言扩展来实现并行处理。本章讲解如何在不同情况下充分利用这些 Python 特性。

第 6 章：内置模块

Python 预装了许多写程序时会用到的重要模块。这些标准软件包与通常意义上的 Python 语言联系得非常紧密，我们可以将其当成语言规范的一部分。本章将会讲解基本的内置模块。

第 7 章：协作开发

如果许多人要开发同一个 Python 程序，那就得仔细商量代码的写法了。即便你是一个人开发，也需要理解其他人所写的模块。本章讲解多人协作开发 Python 程序时所

用的标准工具及最佳做法。

第8章：部署

Python 提供了一些工具，使我们可以把软件部署到不同的环境中。它也提供了一些模块，令开发者可以把程序编写得更加健壮。本章讲解如何使用 Python 调试、优化并测试程序，以提升其质量与性能。

本书使用的约定

本书在 Python 代码风格指南（Python style guide）的基础上做了一些修改，使范例代码便于印刷，也便于凸显其中的重要内容。一行代码比较长时，会以 ➤ 字符来表示折行。代码中的某些部分，与当前要讲的问题联系不大，笔者会将这部分代码略去，并在注释中以省略号来表示（# ...）。为了缩减范例代码的篇幅，笔者也把内嵌的文档删去了。读者在开发自己的项目时不应该这么做，而是应该遵循 Python 风格指南（参见本书第 2 条），并为源代码撰写开发文档（参见本书第 49 条）。

书中大部分代码，运行之后都会产生输出（output）。笔者所谓的输出，意思是说：在交互式解释器（interactive interpreter）中运行这些 Python 程序时，控制台或终端机里面会打印出一些信息。这些打印出来的信息，以等宽字体印刷，它们上方的那一行会标有 >>> 符号（这个 >>> 符号是 Python 解释器的提示符）。笔者使用这个符号是想告诉大家：把 >>> 上方的那些范例代码输入 Python shell 之后，会产生与 >>> 下方文字相符的输出信息。

除此之外，还有一些上方虽无 >>> 符号，但却以等宽字体印刷的代码段。这些内容用来表示产生于 Python 解释器之外的输出信息。它们的上方通常都会有 \$ 字符，这表示笔者是在 Bash 之类的命令行 shell 里面先运行了程序，然后才产生这些输出的。

获取源代码及勘误表

大家可以抛开本书的讲解部分，把某些范例作为完整的程序运行一遍，这样是很好处的。你可以用这些代码做实验，以了解整个程序的运行原理。全部源码都可以从本书网站（<http://www.effectivepython.com/>）下载^①。书中的错误也会张贴到该网站^②。

① 范例代码的网址是：<https://github.com/bslatkin/effectivepython>。——译者注

② 这里针对的是英文原书，勘误表的网址是：<https://github.com/bslatkin/effectivepython/issues>。——译者注

致 谢 *Acknowledgements*

在生活中，有很多人给了我指导、支持及鼓励，没有他们，本书就不会面世。

感谢《Effective Software Development》系列的顾问 Scott Meyers。笔者 15 岁那年初次阅读了 Scott 所写的《Effective C++》，当时我就迷上了这门语言。我后来的教育经历，以及在 Google 的第一份工作，无疑都得益于 Scott 的那本书。这次有机会写作本书，本人深感荣幸。

感谢核心技术评审者 Brett Cannon、Tavis Rudd 和 Mike Taylor，他们为本书提供了深刻而透彻的反馈意见。感谢 Leah Culver 和 Adrian Holovaty，他们两位认为写作这样一本书很有意义。感谢友人 Michael Levine、Marzia Niccolai、Ade Oshineye 和 Katrina Sostek，他们耐心阅读了本书的初稿。也感谢 Google 诸位同事审读本书。若没有以上诸君的帮助，本书读起来可能就会比较费解。

感谢制作本书的每一位工作人员。感谢编辑 Trina MacDonald 启动本书制作流程，并提供大力支持。感谢诸位团队成员帮助制作本书，他们是：策划编辑 Tom Cirtin 和 Chris Zahn、助理编辑 Olivia Basegio、营销经理 Stephane Nakib、文字编辑 Stephanie Geels，以及生产编辑 Julie Nahil。

感谢与我共事的诸位优秀 Python 程序员：Anthony Baxter、Brett Cannon、Wesley Chun、Jeremy Hylton、Alex Martelli、Neal Norwitz、Guido van Rossum、Andy Smith、Greg Stein 和 Ka-Ping Yee。很高兴你们能督促并指引我学习 Python。Python 开发社团构建得非常优秀，成为一名 Python 开发者，令我感到特别荣幸。

感谢诸位同事这些年来对我的关照。感谢 Kevin Gibbs 帮助我应对风险。感谢 Ken Ashcraft、Ryan Barrett 和 Jon McAlister 教会我如何工作。感谢 Brad Fitzpatrick 帮助我提升工作能力。感谢 Paul McDonald 陪我一起创建我们的搞怪项目。感谢 Jeremy Ginsberg

和 Jack Hebert 令其成为现实。

感谢激发我编程兴趣的诸位老师：Ben Chelf、Vince Hugo、Russ Lewin、Jon Stemmler、Derek Thomson 和 Daniel Wang。正因为有了你们的指引，我才会努力磨练编程技术，进而使自己有能力去教导他人。

感谢母亲使我找到了人生的目标并鼓励我做程序员。感谢兄弟、祖父母、众亲戚以及儿时的玩伴，从小你们就是我的榜样，也使我找到了成长的快乐。

最后要感谢我的妻子 Colleen，感谢她的关爱和支持，感谢她带来的欢笑。

目 录 *Contents*

本书赞誉

译者序

前言

致谢

第1章 用 Pythonic 方式来思考..... 1

第1条：确认自己所用的 Python 版本..... 1

第2条：遵循 PEP 8 风格指南..... 3

第3条：了解 bytes、str 与 unicode 的区别..... 5

第4条：用辅助函数来取代复杂的表达式..... 8

第5条：了解切割序列的办法..... 10

第6条：在单次切片操作内，不要同时指定 start、end 和 stride..... 13

第7条：用列表推导来取代 map 和 filter..... 15

第8条：不要使用含有两个以上表达式的列表推导..... 16

第9条：用生成器表达式来改写数据量较大的列表推导..... 18

第10条：尽量用 enumerate 取代 range..... 20

第11条：用 zip 函数同时遍历两个迭代器..... 21

第12条：不要在 for 和 while 循环后面写 else 块..... 23

第13条：合理利用 try/except/else/finally 结构中的每个代码块..... 25

第2章 函数..... 28

第14条：尽量用异常来表示特殊情况，而不要返回 None..... 28

第 15 条：了解如何在闭包里使用外围作用域中的变量	30
第 16 条：考虑用生成器来改写直接返回列表的函数	35
第 17 条：在参数上面迭代时，要多加小心	37
第 18 条：用数量可变的位置参数减少视觉杂讯	41
第 19 条：用关键字参数来表达可选的行为	43
第 20 条：用 None 和文档字符串来描述具有动态默认值的参数	46
第 21 条：用只能以关键字形式指定的参数来确保代码明晰	49
第 3 章 类与继承	53
第 22 条：尽量用辅助类来维护程序的状态，而不要用字典和元组	53
第 23 条：简单的接口应该接受函数，而不是类的实例	58
第 24 条：以 @classmethod 形式的多态去通用地构建对象	62
第 25 条：用 super 初始化父类	67
第 26 条：只在使用 Mix-in 组件制作工具类时进行多重继承	71
第 27 条：多用 public 属性，少用 private 属性	75
第 28 条：继承 collections.abc 以实现自定义的容器类型	79
第 4 章 元类及属性	84
第 29 条：用纯属性取代 get 和 set 方法	84
第 30 条：考虑用 @property 来代替属性重构	88
第 31 条：用描述符来改写需要复用的 @property 方法	92
第 32 条：用 __getattr__、__getattribute__ 和 __setattr__ 实现按需生成的属性	97
第 33 条：用元类来验证子类	102
第 34 条：用元类来注册子类	104
第 35 条：用元类来注解类的属性	108
第 5 章 并发及并行	112
第 36 条：用 subprocess 模块来管理子进程	113
第 37 条：可以用线程来执行阻塞式 I/O，但不要用它做平行计算	117
第 38 条：在线程中使用 Lock 来防止数据竞争	121

第 39 条：用 Queue 来协调各线程之间的工作	124
第 40 条：考虑用协程来并发地运行多个函数	131
第 41 条：考虑用 concurrent.futures 来实现真正的平行计算	141
第 6 章 内置模块	145
第 42 条：用 functools.wraps 定义函数修饰器	145
第 43 条：考虑以 contextlib 和 with 语句来改写可复用的 try/finally 代码	148
第 44 条：用 copyreg 实现可靠的 pickle 操作	151
第 45 条：应该用 datetime 模块来处理本地时间，而不是用 time 模块	157
第 46 条：使用内置算法与数据结构	161
第 47 条：在重视精确度的场合，应该使用 decimal	166
第 48 条：学会安装由 Python 开发者社区所构建的模块	168
第 7 章 协作开发	170
第 49 条：为每个函数、类和模块编写文档字符串	170
第 50 条：用包来安排模块，并提供稳固的 API	174
第 51 条：为自编的模块定义根异常，以便将调用者与 API 相隔离	179
第 52 条：用适当的方式打破循环依赖关系	182
第 53 条：用虚拟环境隔离项目，并重建其依赖关系	187
第 8 章 部署	193
第 54 条：考虑用模块级别的代码来配置不同的部署环境	193
第 55 条：通过 repr 字符串来输出调试信息	195
第 56 条：用 unittest 来测试全部代码	198
第 57 条：考虑用 pdb 实现交互调试	201
第 58 条：先分析性能，然后再优化	203
第 59 条：用 tracemalloc 来掌握内存的使用及泄漏情况	208

用 Pythonic 方式来思考

一门语言的编程习惯是由用户来确立的。这些年来，Python 开发者用 Pythonic 这个形容词来描述那种符合特定风格的代码。这种 Pythonic 风格，既不是非常严密的规范，也不是由编译器强加给开发者的规则，而是大家在使用 Python 语言协同工作的过程中逐渐形成的习惯。Python 开发者不喜欢复杂的事物，他们崇尚直观、简洁而又易读的代码（请在 Python 解释器中输入 `import this`）。

对 C++ 或 Java 等其他语言比较熟悉的人，可能还在按自己喜欢的风格来使用 Python；而刚刚接触 Python 的程序员，则需要逐渐熟悉许多可以用 Python 代码来表达的概念。但无论哪一种开发者，都必须知道如何以最佳方式完成常见的 Python 编程工作，这种最佳方式，就是 Pythonic 方式。该方式将会影响你所写的每个程序。

第 1 条：确认自己所用的 Python 版本

本书绝大部分范例代码都遵循 Python 3.4（发布于 2014 年 3 月 17 日）的语法。某些范例还会同时给出 Python 2.7（发布于 2010 年 7 月 3 日）版本的代码，以强调两者的区别。笔者给出的建议适用于 CPython、Jython、IronPython 及 PyPy 等流行的 Python 运行时环境。

很多电脑都预装了多个版本的标准 CPython 运行时环境^①。然而，在命令行中输入

① 在不引起混淆的情况下，可以把 CPython 理解成默认的 Python 解释器。下同。——译者注

默认的 python 命令之后, 究竟会执行哪个版本则无法肯定。python 通常是 python 2.7 的别名, 但也有可能是 python 2.6 或 python 2.5 等旧版本的别名。请用 --version 标志来运行 python 命令, 以了解所使用的具体 Python 版本。

```
$ python --version
Python 2.7.8
```

通常可以用 python3 命令来运行 Python 3。

```
$ python3 --version
Python 3.4.2
```

运行程序的时候, 也可以在内置的 sys 模块里查询相关的值, 以确定当前使用的 Python 版本。

```
import sys
print(sys.version_info)
print(sys.version)

>>>
sys.version_info(major=3, minor=4, micro=2,
releaselevel='final', serial=0)
3.4.2 (default, Oct 19 2014, 17:52:17)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.51)]
```

Python 2 和 Python 3 都处在 Python 社区的积极维护之中。但是 Python 2 的功能开发已经冻结, 只会进行 bug 修复、安全增强以及移植等工作, 以便使开发者能顺利从 Python 2 迁移到 Python 3。2to3 与 six[Ⓔ]等工具可以帮助大家把代码轻松地适配到 Python 3 及其后续版本上面。

Python 3 经常会添加新功能并提供改进, 而这些功能与改进不会出现在 Python 2 中。笔者写作本书时, 大部分 Python 开源代码库都已经兼容 Python 3 了, 所以强烈建议大家使用 Python 3 来开发自己的下一个 Python 项目。

要点

- ❑ 有两个版本的 Python 处于活跃状态, 它们是: Python 2 与 Python 3。
- ❑ 有很多种流行的 Python 运行时环境, 例如, CPython、Jython、IronPython 以及 PyPy 等。
- ❑ 在操作系统的命令行中运行 Python 时, 请确保该 Python 的版本与你想使用的 Python 版本相符。

[Ⓔ] 该工具的网址是: <https://pythonhosted.org/six/>。——译者注

- ❑ 由于 Python 社区把开发重点放在 Python 3 上，所以在开发后续项目时，应该优先考虑采用 Python 3。

第2条：遵循 PEP 8 风格指南

《Python Enhancement Proposal #8》（8 号 Python 增强提案）又叫 PEP 8，它是针对 Python 代码格式而编订的风格指南。尽管可以在保证语法正确的前提下随意编写 Python 代码，但是，采用一致的风格来书写可以令代码更加易懂、更加易读。采用和其他 Python 程序员相同的风格来写代码，也可以使项目更利于多人协作。即便代码只会由你自己阅读，遵循这套风格也依然可以令后续的修改变得容易一些。

PEP 8 列出了许多细节，以描述如何撰写清晰的 Python 代码。它会随着 Python 语言持续更新。大家应该把整份指南都读一遍（<http://www.python.org/dev/peps/pep-0008>）。下面列出几条绝对应该遵守的规则。

空白：Python 中的空白（whitespace）会影响代码的含义。Python 程序员使用空白的时候尤其在意，因为它们还会影响代码的清晰程度。

- ❑ 使用 space（空格）来表示缩进，而不要用 tab（制表符）。
- ❑ 和语法相关的每一层缩进都用 4 个空格来表示。
- ❑ 每行的字符数不应超过 79。
- ❑ 对于占据多行的长表达式来说，除了首行之外的其余各行都应该在通常的缩进级别之上再加 4 个空格。
- ❑ 文件中的函数与类之间应该用两个空行隔开。
- ❑ 在同一个类中，各方法之间应该用一个空行隔开。
- ❑ 在使用下标来获取列表元素、调用函数或给关键字参数赋值的时候，不要在两旁添加空格。
- ❑ 为变量赋值的时候，赋值符号的左侧和右侧应该各自写上一个空格，而且只写一个就好。

命名：PEP 8 提倡采用不同的命名风格来编写 Python 代码中的各个部分，以便在阅读代码时可以根据这些名称看出它们在 Python 语言中的角色。

- ❑ 函数、变量及属性应该用小写字母来拼写，各单词之间以下划线相连，例如，`lowercase_underscore`。

- ❑ 受保护的实例属性，应该以单个下划线开头，例如，`_leading_underscore`。
- ❑ 私有的实例属性，应该以两个下划线开头，例如，`__double_leading_underscore`。
- ❑ 类与异常，应该以每个单词首字母均大写的形式来命名，例如，`CapitalizedWord`。
- ❑ 模块级别的常量，应该全部采用大写字母来拼写，各单词之间以下划线相连，例如，`ALL_CAPS`。
- ❑ 类中的实例方法（instance method），应该把首个参数命名为 `self`，以表示该对象自身。
- ❑ 类方法（class method）的首个参数，应该命名为 `cls`，以表示该类自身。

表达式和语句：《The Zen of Python》(Python 之禅)中说：“每件事都应该有直白的做法，而且最好只有一种。”PEP 8 在制定表达式和语句的风格时，就试着体现了这种思想。

- ❑ 采用内联形式的否定词，而不要把否定词放在整个表达式的前面，例如，应该写 `if a is not b` 而不是 `if not a is b`。
- ❑ 不要通过检测长度的办法（如 `if len(somelist) == 0`）来判断 `somelist` 是否为 `[]` 或 `"` 等空值，而是应该采用 `if not somelist` 这种写法来判断，它会假定：空值将自动评估为 `False`。
- ❑ 检测 `somelist` 是否为 `[1]` 或 `'hi'` 等非空值时，也应如此，`if somelist` 语句默认会把非空的值判断为 `True`。
- ❑ 不要编写单行的 `if` 语句、`for` 循环、`while` 循环及 `except` 复合语句，而是应该把这些语句分成多行来书写，以示清晰。
- ❑ `import` 语句应该总是放在文件开头。
- ❑ 引入模块的时候，总是应该使用绝对名称，而不应该根据当前模块的路径来使用相对名称。例如，引入 `bar` 包中的 `foo` 模块时，应该完整地写出 `from bar import foo`，而不应该简写为 `import foo`。
- ❑ 如果一定要以相对名称来编写 `import` 语句，那就采用明确的写法：`from . import foo`。
- ❑ 文件中的那些 `import` 语句应该按顺序划分成三个部分，分别表示标准库模块、第三方模块以及自用模块。在每一部分之中，各 `import` 语句应该按模块的字母顺序来排列。



注意 Pylint (<http://www.pylint.org/>) 是一款流行的 Python 源码静态分析工具。它可以自动检查受测代码是否符合 PEP 8 风格指南，而且还能找出 Python 程序里的多种常见错误。

要点

- 当编写 Python 代码时，总是应该遵循 PEP 8 风格指南。
- 与广大 Python 开发者采用同一套代码风格，可以使项目更利于多人协作。
- 采用一致的风格来编写代码，可以令后续的修改工作变得更为容易。

第3条：了解 bytes、str 与 unicode 的区别

Python 3 有两种表示字符序列的类型：bytes 和 str。前者的实例包含原始的 8 位值^①；后者的实例包含 Unicode 字符。

Python 2 也有两种表示字符序列的类型，分别叫做 str 和 unicode。与 Python 3 不同的是，str 的实例包含原始的 8 位值；而 unicode 的实例，则包含 Unicode 字符。

把 Unicode 字符表示为二进制数据（也就是原始 8 位值）有许多种办法。最常见的编码方式就是 UTF-8。但是大家要记住，Python 3 的 str 实例和 Python 2 的 unicode 实例都没有和特定的二进制编码形式相关联。要想把 Unicode 字符转换成二进制数据，就必须使用 encode 方法。要想把二进制数据转换成 Unicode 字符，则必须使用 decode 方法。

编写 Python 程序的时候，一定要把编码和解码操作放在界面最外围来做。程序的核心部分应该使用 Unicode 字符类型（也就是 Python 3 中的 str、Python 2 中的 unicode），而且不要对字符编码做任何假设。这种办法既可以令程序接受多种类型的文本编码（如 Latin-1、Shift JIS 和 Big5），又可以保证输出的文本信息只采用一种编码形式（最好是 UTF-8）。

由于字符类型有别，所以 Python 代码中经常会出现两种常见的使用情境：

- 开发者需要原始 8 位值，这些 8 位值表示以 UTF-8 格式（或其他编码形式）来编码的字符。
- 开发者需要操作没有特定编码形式的 Unicode 字符。

所以，我们需要编写两个辅助（helper）函数，以便在这两种情况之间转换，使得转换后的输入数据能够符合开发者的预期。

在 Python 3 中，我们需要编写接受 str 或 bytes，并总是返回 str 的方法：

① 就是原始的字节，由于每个字节有 8 个二进制位，所以是原始的 8 位值。也叫做原生 8 位值、纯 8 位值。——译者注

```
def to_str(bytes_or_str):
    if isinstance(bytes_or_str, bytes):
        value = bytes_or_str.decode('utf-8')
    else:
        value = bytes_or_str
    return value # Instance of str
```

另外，还需要编写接受 str 或 bytes，并总是返回 bytes 的方法：

```
def to_bytes(bytes_or_str):
    if isinstance(bytes_or_str, str):
        value = bytes_or_str.encode('utf-8')
    else:
        value = bytes_or_str
    return value # Instance of bytes
```

在 Python 2 中，需要编写接受 str 或 unicode，并总是返回 unicode 的方法：

```
# Python 2
def to_unicode(unicode_or_str):
    if isinstance(unicode_or_str, str):
        value = unicode_or_str.decode('utf-8')
    else:
        value = unicode_or_str
    return value # Instance of unicode
```

另外，还需要编写接受 str 或 unicode，并总是返回 str 的方法：

```
# Python 2
def to_str(unicode_or_str):
    if isinstance(unicode_or_str, unicode):
        value = unicode_or_str.encode('utf-8')
    else:
        value = unicode_or_str
    return value # Instance of str
```

在 Python 中使用原始 8 位值与 Unicode 字符时，有两个问题要注意。

第一个问题可能会出现在 Python 2 里面。如果 str 只包含 7 位 ASCII 字符，那么 unicode 和 str 实例似乎就成了同一种类型。

- ❑ 可以用 + 操作符把这种 str 与 unicode 连接起来。
- ❑ 可以用等价与不等价操作符，在这种 str 实例与 unicode 实例之间进行比较。
- ❑ 在格式字符串中，可以用 '%s' 等形式来代表 unicode 实例。

这些行为意味着，在只处理 7 位 ASCII 的情境下，如果某函数接受 str，那么可以给它传入 unicode；如果某函数接受 unicode，那么也可以给它传入 str。而在 Python 3 中，bytes 与 str 实例则绝对不会等价，即使是空字符串也不行。所以，在传入字符序列的时候必须留意其类型。

第二个问题可能会出现在 Python 3 里面。如果通过内置的 `open` 函数获取了文件句柄^①，那么请注意，该句柄默认会采用 UTF-8 编码格式来操作文件。而在 Python 2 中，文件操作的默认编码格式则是二进制形式。这可能会导致程序出现奇怪的错误，对习惯了 Python 2 的程序员来说更是如此。

例如，现在要向文件中随机写入一些二进制数据。下面这种用法在 Python 2 中可以正常运作，但在 Python 3 中不行。

```
with open('/tmp/random.bin', 'w') as f:
    f.write(os.urandom(10))
```

```
>>>
TypeError: must be str, not bytes
```

发生上述异常的原因在于，Python 3 给 `open` 函数添加了名为 `encoding` 的新参数，而这个新参数的默认值却是 `'utf-8'`。这样在文件句柄上进行 `read` 和 `write` 操作时，系统就要求开发者必须传入包含 Unicode 字符的 `str` 实例，而不接受包含二进制数据的 `bytes` 实例。

为了解决这个问题，我们必须用二进制写入模式 (`'wb'`) 来开启待操作的文件，而不能像原来那样，采用字符写入模式 (`'w'`)。按照下面这种方式来使用 `open` 函数，即可同时适配 Python 2 与 Python 3：

```
with open('/tmp/random.bin', 'wb') as f:
    f.write(os.urandom(10))
```

从文件中读取数据的时候也有这种问题。解决办法与写入时相似：用 `'rb'` 模式（也就是二进制模式）打开文件，而不要使用 `'r'` 模式。

要点

- ❑ 在 Python 3 中，`bytes` 是一种包含 8 位值的序列，`str` 是一种包含 Unicode 字符的序列。开发者不能以 `>` 或 `+` 等操作符来混同操作 `bytes` 和 `str` 实例。
- ❑ 在 Python 2 中，`str` 是一种包含 8 位值的序列，`unicode` 是一种包含 Unicode 字符的序列。如果 `str` 只含有 7 位 ASCII 字符，那么可以通过相关的操作符来同时使用 `str` 与 `unicode`。
- ❑ 在对输入的数据进行操作之前，使用辅助函数来保证字符序列的类型与开发者的期望相符（有的时候，开发者想操作以 UTF-8 格式来编码的 8 位值，有的时候，则想操作 Unicode 字符）。

① 文件句柄（file handle）其实是一种标识符或指针，也可以理解为文件描述符，用来指代开发者将要操作的文件。本书会酌情按照习惯称呼，将 `handle` 一词译为句柄、操作标识或描述符。——译者注

❑ 从文件中读取二进制数据，或向其中写入二进制数据时，总应该以 'rb' 或 'wb' 等二进制模式来开启文件。

第 4 条：用辅助函数来取代复杂的表达式

Python 的语法非常精练，很容易就能用一行表达式来实现许多逻辑。例如，要从 URL 中解码查询字符串。在下例所举的查询字符串中，每个参数都可以表示一个整数值：

```
from urllib.parse import parse_qs
my_values = parse_qs('red=5&blue=0&green=',
                    keep_blank_values=True)
print(repr(my_values))

>>>
{'red': ['5'], 'green': ['',], 'blue': ['0']}
```

查询字符串中的某些参数可能有多个值，某些参数可能只有一个值，某些参数可能是空白 (blank) 值，还有些参数则没有出现在查询字符串之中。用 get 方法在 my_values 字典中查询不同的参数时，就有可能获得不同的返回值。

```
print('Red:      ', my_values.get('red'))
print('Green:    ', my_values.get('green'))
print('Opacity:  ', my_values.get('opacity'))

>>>
Red:      ['5']
Green:    ['',]
Opacity:  None
```

如果待查询的参数没有出现在字符串中，或当该参数的值为空白时能够返回默认值 0，那就更好了。这个逻辑看上去似乎并不值得用完整的 if 语句或辅助函数来实现，于是，你可能会考虑用 Boolean 表达式。

由于 Python 的语法非常精练，所以我们很容易就想到了这种做法。空字符串、空列表及零值，都会评估为 False。因此，在下面这个例子中，如果 or 操作符左侧的子表达式估值为 False，那么整个表达式的值就将是 or 操作符右侧那个子表达式的值。

```
# For query string 'red=5&blue=0&green='
red = my_values.get('red', ['',])[0] or 0
green = my_values.get('green', ['',])[0] or 0
opacity = my_values.get('opacity', ['',])[0] or 0
print('Red:      %r' % red)
print('Green:    %r' % green)
print('Opacity:  %r' % opacity)
```



```
>>>
Red:      '5'
Green:    0
Opacity: 0
```

red 那一行代码是正确的，因为 `my_values` 字典里确实有 'red' 这个键。该键所对应的值是个列表，列表中只有一个元素，也就是字符串 '5'。这个字符串会自动估值为 True，所以，or 表达式第一部分的值就会赋给 red。

green 那一行代码也是正确的，因为 `get` 方法从 `my_values` 字典中获取的值是个列表，该列表只有一个元素，这个元素是个空字符串。由于空字符串会自动估值为 False，所以整个 or 表达式的值就成了 0。

opacity 那一行代码也没有错。`my_values` 字典里面没有名为 'opacity' 的键，而当字典中没有待查询的键时，`get` 方法会返回第二个参数的值，所以，在本例中，`get` 方法就会返回仅包含一个元素的列表，那个元素是个空字符串。当字典里没有待查询的 'opacity' 键时，这行代码的执行效果与 green 那行代码相同。

这样的长表达式虽然语法正确，但却很难阅读，而且有时也未必完全符合要求。由于我们想在数学表达式中使用这些参数值，所以还要确保每个参数的值都是整数。为了实现这一点，我们需要把每个长表达式都包裹在内置的 `int` 函数中，以便把字符串解析为整数。

```
red = int(my_values.get('red', ['']))[0] or 0)
```

这种写法读起来很困难，而且看上去很乱。这样的代码不容易理解，初次拿到这种代码的人，可能先要花些功夫把表达式拆解开，然后才能看明白它的作用。即使想把代码写得简省一些，也没有必要将全部内容都挤在一行里面。

Python 2.5 添加了 if/else 条件表达式（又称三元操作符），使我们可以把上述逻辑写得清晰一些，同时还能保持代码简洁。

```
red = my_values.get('red', [''])
red = int(red[0]) if red[0] else 0
```

这种写法比原来好了一些。对于不太复杂的情况来说，if/else 条件表达式可以令代码变得清晰。但对于上面这个例子来说，它的清晰程度还是比不上跨越多行的完整 if/else 语句。如果把上述逻辑全都改成下面这种形式，那我们就能就感觉到：刚才那种紧缩的写法其实挺复杂的。

```
green = my_values.get('green', [''])
if green[0]:
```

```

    green = int(green[0])
else:
    green = 0

```

现在应该把它总结成辅助函数了，如果需要频繁使用这种逻辑，那就更应该这样做。

```

def get_first_int(values, key, default=0):
    found = values.get(key, [''])
    if found[0]:
        found = int(found[0])
    else:
        found = default
    return found

```

调用这个辅助函数时所使用的代码，要比使用 or 操作符的长表达式版本，以及使用 if/else 表达式的两行版本更加清晰。

```
green = get_first_int(my_values, 'green')
```

表达式如果变得比较复杂，那就应该考虑将其拆解成小块，并把这些逻辑移入辅助函数中。这会令代码更加易读，它比原来那种密集的写法更好。编写 Python 程序时，不要一味追求过于紧凑的写法，那样会写出非常复杂的表达式。

要点

- ❑ 开发者很容易过度运用 Python 的语法特性，从而写出那种特别复杂并且难以理解的单行表达式。
- ❑ 请把复杂的表达式移入辅助函数之中，如果要反复使用相同的逻辑，那就更应该这么做。
- ❑ 使用 if/else 表达式，要比用 or 或 and 这样的 Boolean 操作符写成的表达式更加清晰。

第 5 条：了解切割序列的办法

Python 提供了一种把序列切成小块的写法。这种切片（slice）^①操作，使得开发者能够轻易地访问由序列中的某些元素所构成的子集。最简单的用法，就是对内置的 list、str 和 bytes 进行切割。切割操作还可以延伸到实现了 `__getitem__` 和 `__setitem__` 这两个特殊方法的 Python 类上（参见本书第 28 条）。

① 在不引起混淆时，本书也将其称为切割，下同。——译者注

切割操作的基本写法是 `somelist[start:end]`，其中 `start`（起始索引）所指的元素涵盖在切割后的范围内，而 `end`（结束索引）所指的元素则不包括在切割结果之中。

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
print('First four:', a[:4])
print('Last four: ', a[-4:])
print('Middle two:', a[3:-3])

>>>
First four: ['a', 'b', 'c', 'd']
Last four:  ['e', 'f', 'g', 'h']
Middle two: ['d', 'e']
```

如果从列表开头获取切片，那就不要在 `start` 那里写上 0，而是应该把它留空，这样代码看起来会清爽一些。

```
assert a[:5] == a[0:5]
```

如果切片一直要取到列表末尾，那就应该把 `end` 留空，因为即便写了，也是多余。

```
assert a[5:] == a[5:len(a)]
```

在指定切片起止索引时，若要从列表尾部向前算，则可使用负值来表示相关偏移量。如果采用下面这些写法来切割列表，那么即便是刚刚接触代码的人也能立刻明白程序的意图。由于这些写法都不会令人惊讶，所以笔者推荐大家在代码中放心地使用。

```
a[:]      # ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
a[:5]     # ['a', 'b', 'c', 'd', 'e']
a[:-1]    # ['a', 'b', 'c', 'd', 'e', 'f', 'g']
a[4:]     #                ['e', 'f', 'g', 'h']
a[-3:]    #                ['f', 'g', 'h']
a[2:5]    #                ['c', 'd', 'e']
a[2:-1]   #                ['c', 'd', 'e', 'f', 'g']
a[-3:-1]  #                ['f', 'g']
```

切割列表时，即便 `start` 或 `end` 索引越界也不会出问题。利用这一特性，我们可以限定输入序列的最大长度^①。

```
first_twenty_items = a[:20]
last_twenty_items = a[-20:]
```

反之，访问列表中的单个元素时，下标不能越界，否则会导致异常。

```
a[20]

>>>
IndexError: list index out of range
```

① 也就是只取该序列开头的 `n` 个值或末尾的 `n` 个值。——译者注



注意 请注意，如果使用负变量作为 **start** 索引来切割列表，那么在极个别情况下，可能会导致奇怪的结果。例如，`somelist[-n:]` 这个表达式，在 `n` 大于 1 时可以正常运作[⊖]，如当 `n` 为 3 时，`somelist[-3:]` 的结果是正常的。然而，当 `n` 为 0 时，表达式 `somelist[-0:]` 则成了原列表的一份拷贝。

对原列表进行切割之后，会产生另外一份全新的列表。系统依然维护着指向原列表中各个对象的引用。在切割后得到的新列表上进行修改，不会影响原列表。

```
b = a[4:]
print('Before: ', b)
b[1] = 99
print('After: ', b)
print('No change:', a)

>>>
Before:  ['e', 'f', 'g', 'h']
After:   ['e', 99, 'g', 'h']
No change: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

在赋值时对左侧列表使用切割操作，会把该列表中处在指定范围内的对象替换为新值。与元组 (tuple) 的赋值（如 `a, b = c[:2]`）不同，此切片的长度无需新值的个数相等。位于切片范围之前及之后的那些值都保留不变。列表会根据新值的个数相应地扩张或收缩。

```
print('Before ', a)
a[2:7] = [99, 22, 14]
print('After ', a)

>>>
Before  ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
After   ['a', 'b', 99, 22, 14, 'h']
```

如果对赋值操作右侧的列表使用切片，而把切片的起止索引都留空，那就会产生一份原列表的拷贝。

```
b = a[:]
assert b == a and b is not a
```

如果对赋值操作左侧的列表使用切片，而又没有指定起止索引，那么系统会把右侧的新值复制一份，并用这份拷贝来替换左侧列表的全部内容，而不会重新分配新的列表。

```
b = a
print('Before', a)
a[:] = [101, 102, 103]
```

⊖ 当 `n` 等于 1 时，也可以正常运作。——译者注

```

assert a is b          # Still the same list object
print('After ', a)     # Now has different contents

>>>
Before ['a', 'b', 99, 22, 14, 'h']
After [101, 102, 103]

```

要点

- ❑ 不要写多余的代码：当 start 索引为 0，或 end 索引为序列长度时，应该将其省略。
- ❑ 切片操作不会计较 start 与 end 索引是否越界，这使得我们很容易就能从序列的前端或后端开始，对其进行范围固定的切片操作（如 `a[:20]` 或 `a[-20:]`）。
- ❑ 对 list 赋值的时候，如果使用切片操作，就会把原列表中处在相关范围内的值替换成新值，即便它们的长度不同也依然可以替换。

第 6 条：在单次切片操作内，不要同时指定 start、end 和 stride

除了基本的切片操作（参见本书第 5 条）之外，Python 还提供了 `somelist[start:end:stride]` 形式的写法，以实现步进式切割，也就是从每 n 个元素里面取 1 个出来。例如，可以指定步进值（stride），把列表中位于偶数索引处和奇数索引处的元素分成两组：

```

a = ['red', 'orange', 'yellow', 'green', 'blue', 'purple']
odds = a[::2]
evens = a[1::2]
print(odds)
print(evens)

>>>
['red', 'yellow', 'blue']
['orange', 'green', 'purple']

```

问题在于，采用 stride 方式进行切片时，经常会出现不符合预期的结果。例如，Python 中有一种常见的技巧，能够把以字节形式存储的字符串反转过来，这个技巧就是采用 `-1` 做步进值。

```

x = b'mongoose'
y = x[::-1]
print(y)

>>>
b'esooognom'

```

这种技巧对字节串和 ASCII 字符有用，但是对已经编码成 UTF-8 字节串的 Unicode

字符来说, 则无法奏效。

```
w = '謝謝'
x = w.encode('utf-8')
y = x[::-1]
z = y.decode('utf-8')

>>>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xd9 in
position 0: invalid start byte
```

除了 -1 之外, 其他的负步进值有没有意义呢? 请看下面的例子。

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
a[::2]    # ['a', 'c', 'e', 'g']
a[::-2]   # ['h', 'f', 'd', 'b']
```

上例中, ::2 表示从头部开始, 每两个元素选取一个。::-2 则表示从尾部开始, 向前选取, 每两个元素里选一个。

2::2 是什么意思? -2::-2、-2:2:-2 和 2:2:-2 又是什么意思? 请看下面的例子。

```
a[2::2]    # ['c', 'e', 'g']
a[-2::-2]  # ['g', 'e', 'c', 'a']
a[-2:2:-2] # ['g', 'e']
a[2:2:-2]  # []
```

通过上面几个例子可以看出: 切割列表时, 如果指定了 stride, 那么代码可能会变得相当费解。在一对中括号里写上 3 个数字显得太过拥挤, 从而导致代码难以阅读。这种写法使得 start 和 end 索引的含义变得模糊, 当 stride 为负值时, 尤其如此。

为了解决这种问题, 我们不应该把 stride 与 start 和 end 写在一起。如果非要用 stride, 那就尽量采用正值, 同时省略 start 和 end 索引。如果一定要配合 start 或 end 索引来使用 stride, 那么请考虑先做步进式切片, 把切割结果赋给某个变量, 然后在那个变量上面做第二次切割。

```
b = a[::2]    # ['a', 'c', 'e', 'g']
c = b[1:-1]   # ['c', 'e']
```

上面这种先做步进切割, 再做范围切割的办法[⊖], 会多产生一份原数据的浅拷贝。执行第一次切割操作时, 应该尽量缩减切割后的列表尺寸。如果你所开发的程序对执行时间或内存用量的要求非常严格, 以致不能采用两阶段切割法, 那就请考虑 Python 内置的 itertools 模块。该模块中有个 islide 方法, 这个方法不允许为 start、end 或 stride 指定负值 (参见本书第 46 条)。

⊖ 也可以先做范围切割, 再做步进切割。例如, b = a[2:6]; c = b[::2]。——译者注

要点

- 既有 start 和 end，又有 stride 的切割操作，可能会令人费解。
- 尽量使用 stride 为正数，且不带 start 或 end 索引的切割操作。尽量避免用负数做 stride。
- 在同一个切片操作内，不要同时使用 start、end 和 stride。如果确实需要执行这种操作，那就考虑将其拆解为两条赋值语句，其中一条做范围切割，另一条做步进切割，或考虑使用内置 itertools 模块中的 islice。

第7条：用列表推导来取代 map 和 filter

Python 提供了一种精练的写法，可以根据一份列表来制作另外一份。这种表达式称为 *list comprehension*（列表推导）。例如，要用列表中每个元素的平方值构建另一份列表。如果采用列表推导来实现，那就同时指定制作新列表时所要迭代的输入序列，以及计算新列表中每个元素的值时所用的表达式。

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [x**2 for x in a]
print(squares)

>>>
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

除非是调用只有一个参数的函数，否则，对于简单的情况来说，列表推导要比内置的 map 函数更清晰。如果使用 map，那就要创建 lambda 函数，以便计算新列表中各个元素的值，这会使代码看起来有些乱。

```
squares = map(lambda x: x ** 2, a)
```

列表推导则不像 map 那么复杂，它可以直接过滤原列表中的元素，使得生成的新列表不会包含对应的计算结果。例如，在计算平方值时，我们只想计算那些可以为 2 所整除的数。如果采用列表推导来做，那么只需在循环后面添加条件表达式即可：

```
even_squares = [x**2 for x in a if x % 2 == 0]
print(even_squares)

>>>
[4, 16, 36, 64, 100]
```

把内置的 filter 函数与 map 结合起来，也能达成同样的效果，但是代码会写得非常难懂。

```
alt = map(lambda x: x**2, filter(lambda x: x % 2 == 0, a))
assert even_squares == list(alt)
```

字典 (dict) 与集 (set)，也有和列表类似的推导机制。编写算法时，可以通过这些推导机制来创建衍生的数据结构。

```
chile_ranks = {'ghost': 1, 'habanero': 2, 'cayenne': 3}
rank_dict = {rank: name for name, rank in chile_ranks.items()}
chile_len_set = {len(name) for name in rank_dict.values()}
print(rank_dict)
print(chile_len_set)

>>>
{1: 'ghost', 2: 'habanero', 3: 'cayenne'}
{8, 5, 7}
```

要点

- ❑ 列表推导要比内置的 map 和 filter 函数清晰，因为它无需额外编写 lambda 表达式。
- ❑ 列表推导可以跳过输入列表中的某些元素，如果改用 map 来做，那就必须辅以 filter 方能实现。
- ❑ 字典与集也支持推导表达式。

第 8 条：不要使用含有两个以上表达式的列表推导

除了基本的用法（参见本书第 7 条）之外，列表推导也支持多重循环。例如，要把矩阵（也就是包含列表的列表，即二维列表）简化成一维列表，使原来的每个单元格都成为新列表中的普通元素。这个功能采用包含两个 for 表达式的列表推导即可实现，这些 for 表达式会按照从左至右的顺序来评估。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [x for row in matrix for x in row]
print(flat)

>>>
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

上面这个例子简单易懂，这就是多重循环的合理用法。还有一种包含多重循环的合理用法，那就是根据输入列表来创建有两层深度的新列表。例如，我们要对二维矩阵中的每个单元格取平方，然后用这些平方值构建新的矩阵。由于要多使用一对中括号，所

以实现该功能的代码会比上例稍微复杂一点，但是依然不难理解。

```
squared = [[x**2 for x in row] for row in matrix]
print(squared)
```

```
>>>
[[1, 4, 9], [16, 25, 36], [49, 64, 81]]
```

如果表达式里还有一层循环，那么列表推导就会变得很长，这时必须把它分成多行来写，才能看得清楚一些。

```
my_lists = [
    [[1, 2, 3], [4, 5, 6]],
    # ...
]
flat = [x for sublist1 in my_lists
        for sublist2 in sublist1
        for x in sublist2]
```

可以看出，此时的列表推导并没有比普通的写法更加简洁。于是，笔者改用普通的循环语句来实现相同的效果。由于循环语句带有适当的缩进，所以看上去要比列表推导更清晰。

```
flat = []
for sublist1 in my_lists:
    for sublist2 in sublist1:
        flat.extend(sublist2)
```

列表推导也支持多个 if 条件。处在同一循环级别中的多项条件，彼此之间默认形成 and 表达式。例如，要从数字列表中选出大于 4 的偶数，那么下面这两种列表推导方式是等效的。

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
b = [x for x in a if x > 4 if x % 2 == 0]
c = [x for x in a if x > 4 and x % 2 == 0]
```

每一级循环的 for 表达式后面都可以指定条件。例如，要从原矩阵中把那些本身能为 3 所整除，且其所在行的各元素之和又大于等于 10 的单元格挑出来。我们只需编写很简短的代码，就可用列表推导来实现此功能，但是，这样的代码非常难懂。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
filtered = [[x for x in row if x % 3 == 0]
            for row in matrix if sum(row) >= 10]
print(filtered)

>>>
[[6], [9]]
```

尽管这个例子稍微有点复杂，但在实际编程中，确实会出现这种看上去似乎适合用列表推导来实现的情况。笔者强烈建议大家尽量不要编写这种包含复杂式子的列表推导。这样会使其他人很难理解这段代码。这么写虽然能省下几行空间，但却会给稍后阅读代码的人带来很大障碍。

在列表推导中，最好不要使用两个以上的表达式。可以使用两个条件、两个循环或一个条件搭配一个循环。如果要写的代码比这还复杂，那就应该使用普通的 if 和 for 语句，并编写辅助函数（参见本书第 16 条）。

要点

- ❑ 列表推导支持多级循环，每一级循环也支持多项条件。
- ❑ 超过两个表达式的列表推导是很难理解的，应该尽量避免。

第 9 条：用生成器表达式来改写数据量较大的列表推导

列表推导（参见本书第 7 条）的缺点是：在推导过程中，对于输入序列中的每个值来说，可能都要创建仅含一项元素的全新列表。当输入的数据比较少时，不会出问题，但如果输入的数据非常多，那么可能会消耗大量内存，并导致程序崩溃。

例如，要读取一份文件并返回每行的字符数。若采用列表推导来做，则需把文件每一行的长度都保存在内存中。如果这个文件特别大，或是通过无休止的 network socket（网络套接字）来读取，那么这种列表推导就会出问题。下面的这段列表推导代码，只适合处理少量的输入值。

```
value = [len(x) for x in open('/tmp/my_file.txt')]
print(value)

>>>
[100, 57, 15, 1, 12, 75, 5, 86, 89, 11]
```

为了解决此问题，Python 提供了生成器表达式（*generator expression*），它是对列表推导和生成器的一种泛化（*generalization*）。生成器表达式在运行的时候，并不会把整个输出序列都呈现出来，而是会估值为迭代器（*iterator*），这个迭代器每次可以根据生成器表达式产生一项数据。

把实现列表推导所用的那种写法放在一对圆括号中，就构成了生成器表达式。下面给出的生成器表达式与刚才的代码等效。二者的区别在于，对生成器表达式求值的时

候，它会立刻返回一个迭代器，而不会深入处理文件中的内容。

```
it = (len(x) for x in open('/tmp/my_file.txt'))
print(it)

>>>
<generator object <genexpr> at 0x101b81480>
```

以刚才返回的那个迭代器为参数，逐次调用内置的 `next` 函数，即可使其按照生成器表达式来输出下一个值。可以根据自己的需要，多次命令迭代器根据生成器表达式来生成新值，而不用担心内存用量激增。

```
print(next(it))
print(next(it))

>>>
100
57
```

使用生成器表达式还有个好处，就是可以互相组合。下面这行代码会把刚才那个生成器表达式所返回的迭代器用作另外一个生成器表达式的输入值。

```
roots = ((x, x**0.5) for x in it)
```

外围的迭代器每次前进时，都会推动内部那个迭代器，这就产生了连锁效应，使得执行循环、评估条件表达式、对接输入和输出等逻辑都组合在了一起。

```
print(next(roots))

>>>
(15, 3.872983346207417)
```

上面这种连锁生成器表达式，可以迅速在 Python 中执行。如果要把多种手法组合起来，以操作大批量的输入数据，那最好是用生成器表达式来实现。只是要注意：由生成器表达式所返回的那个迭代器是有状态的，用过一轮之后，就不要反复使用了（参见本书第 17 条）。

要点

- ❑ 当输入的数据量较大时，列表推导可能会因为占用太多内存而出问题。
- ❑ 由生成器表达式所返回的迭代器，可以逐次产生输出值，从而避免了内存用量问题。
- ❑ 把某个生成器表达式所返回的迭代器，放在另一个生成器表达式的 `for` 子表达式中，即可将二者组合起来。
- ❑ 串在一起的生成器表达式执行速度很快。

第 10 条：尽量用 `enumerate` 取代 `range`

在一系列整数上面迭代时，内置的 `range` 函数很有用。

```
random_bits = 0
for i in range(64):
    if randint(0, 1):
        random_bits |= 1 << i
```

对于字符串列表这样的序列式数据结构，可以直接在上面迭代。

```
flavor_list = ['vanilla', 'chocolate', 'pecan', 'strawberry']
for flavor in flavor_list:
    print('%s is delicious' % flavor)
```

当迭代列表的时候，通常还想知道当前元素在列表中的索引。例如，要按照喜好程度打印出自己爱吃的冰淇淋口味。一种办法是用 `range` 来做。

```
for i in range(len(flavor_list)):
    flavor = flavor_list[i]
    print('%d: %s' % (i + 1, flavor))
```

与单纯迭代 `flavor_list` 或是单纯使用 `range` 的代码相比，上面这段代码有些生硬。我们必须获取列表长度，并且通过下标来访问数组^①。这种代码不便于理解。

Python 提供了内置的 `enumerate` 函数，以解决此问题。`enumerate` 可以把各种迭代器^②包装为生成器，以便稍后产生输出值。生成器每次产生一对输出值，其中，前者表示循环下标，后者表示从迭代器中获取到的下一个序列元素。这样写出来的代码会非常简洁。

```
for i, flavor in enumerate(flavor_list):
    print('%d: %s' % (i + 1, flavor))

>>>
1: vanilla
2: chocolate
3: pecan
4: strawberry
```

还可以直接指定 `enumerate` 函数开始计数时所用的值（本例从 1 开始计数），这样能把代码写得更短。

```
for i, flavor in enumerate(flavor_list, 1):
    print('%d: %s' % (i, flavor))
```

① 这里的数组是一种泛称，凡是可以由递增的非负整数做下标来访问其元素的那些数据结构都不妨视为数组，并不一定专指狭义的 `array`。下同。——译者注

② 也包括各种序列以及各种支持迭代的对象，比如，本例中的 `flavor_list`。——译者注

要点

- ❑ `enumerate` 函数提供了一种精简的写法，可以在遍历迭代器时获知每个元素的索引。
- ❑ 尽量用 `enumerate` 来改写那种将 `range` 与下标访问相结合的序列遍历代码。
- ❑ 可以给 `enumerate` 提供第二个参数，以指定开始计数时所用的值（默认为 0）。

第 11 条：用 `zip` 函数同时遍历两个迭代器

在编写 Python 代码时，我们通常要面对很多列表，而这些列表里的对象，可能也是相互关联的。通过列表推导，很容易就能根据某个表达式从源列表推算出一份派生列表（参见本书第 7 条）。

```
names = ['Cecilia', 'Lise', 'Marie']
letters = [len(n) for n in names]
```

对于本例中的派生列表和源列表来说，相同索引处的两个元素之间有着关联。如果想平行地迭代这两份列表，那么可以根据 `names` 源列表的长度来执行循环。

```
longest_name = None
max_letters = 0

for i in range(len(names)):
    count = letters[i]
    if count > max_letters:
        longest_name = names[i]
        max_letters = count
```

```
print(longest_name)
```

```
>>>
Cecilia
```

上面这段代码的问题在于，整个循环语句看上去很乱。用下标来访问 `names` 和 `letters` 会使代码不易阅读。用循环下标 `i` 来访问数组的写法一共出现了两次。改用 `enumerate` 来做（参见本书第 10 条）可以稍稍缓解这个问题，但仍然不够理想。

```
for i, name in enumerate(names):
    count = letters[i]
    if count > max_letters:
        longest_name = name
        max_letters = count
```

使用 Python 内置的 `zip` 函数，能够令上述代码变得更为简洁。在 Python 3 中的 `zip` 函数，可以把两个或两个以上的迭代器封装为生成器，以便稍后求值。这种 `zip` 生成

器，会从每个迭代器中获取该迭代器的下一个值，然后把这些值汇聚成元组（tuple）。与通过下标来访问多份列表的那种写法相比，这种用 zip 写出来的代码更加明晰。

```
for name, count in zip(names, letters):
    if count > max_letters:
        longest_name = name
        max_letters = count
```

内置的 zip 函数有两个问题。

第一个问题是，Python 2 中的 zip 并不是生成器，而是会把开发者所提供的那些迭代器，都平行地遍历一次，在此过程中，它都会把那些迭代器所产生的值汇聚成元组，并把那些元组所构成的列表完整地返回给调用者。这可能会占用大量内存并导致程序崩溃。如果要在 Python 2 里用 zip 来遍历数据量非常大的迭代器，那么应该使用 itertools 内置模块中的 izip 函数（参见本书第 46 条）。

第二个问题是，如果输入的迭代器长度不同，那么 zip 会表现出奇怪的行为。例如，我们又给 names 里添加了一个名字，但却忘了把这个名字的字母数量更新到 letters 之中。现在，如果用 zip 同时遍历这两份列表，那就会产生意外的结果。

```
names.append('Rosalind')
for name, count in zip(names, letters):
    print(name)

>>>
Cecilia
Lise
Marie
```

新元素 'Rosalind' 并没有出现在遍历结果中。这正是 zip 的运作方式。受封装的那些迭代器中，只要有一个耗尽，zip 就不再产生元组了。如果待遍历的迭代器长度都相同，那么这种运作方式不会出问题，由列表推导所推算出的派生列表一般都和源列表等长。如果待遍历的迭代器长度不同，那么 zip 会提前终止，这将会导致意外的结果。若不能确定 zip 所封装的列表是否等长，则可考虑改用 itertools 内置模块中的 zip_longest 函数（此函数在 Python 2 里叫做 izip_longest）。

要点

- 内置的 zip 函数可以平行地遍历多个迭代器。
- Python 3 中的 zip 相当于生成器，会在遍历过程中逐次产生元组，而 Python 2 中的 zip 则是直接把这些元组完全生成好，并一次性地返回整份列表。

- ❑ 如果提供的迭代器长度不等，那么 `zip` 就会自动提前终止。
- ❑ `itertools` 内置模块中的 `zip_longest` 函数可以平行地遍历多个迭代器，而不用在乎它们的长度是否相等（参见本书第 46 条）。

第 12 条：不要在 `for` 和 `while` 循环后面写 `else` 块

Python 提供了一种很多编程语言都不支持的功能，那就是可以在循环内部的语句块后面直接编写 `else` 块。

```
for i in range(3):
    print('Loop %d' % i)
else:
    print('Else block!')
```

```
>>>
Loop 0
Loop 1
Loop 2
Else block!
```

奇怪的是，这种 `else` 块会在整个循环执行完之后立刻运行。既然如此，那它为什么叫做 `else` 呢？为什么不叫 `and`？在 `if/else` 语句中，`else` 的意思是：如果不执行前面那个 `if` 块，那就执行 `else` 块。在 `try/except` 语句中，`except` 的定义也类似：如果前面那个 `try` 块没有成功执行，那就执行 `except` 块。

同理，`try/except/else` 也是如此（参见本书第 13 条），该结构的 `else` 的含义是：如果前面的 `try` 块没有失败，那就执行 `else` 块。`try/finally` 同样非常直观，这里的 `finally` 的意思是：执行过前面的 `try` 块之后，总是执行 `finally` 块。

明白了 `else`、`except` 和 `finally` 的含义之后，刚接触 Python 的程序员可能会把 `for/else` 结构中的 `else` 块理解为：如果循环没有正常执行完，那就执行 `else` 块。实际上刚好相反——在循环里用 `break` 语句提前跳出，会导致程序不执行 `else` 块。

```
for i in range(3):
    print('Loop %d' % i)
    if i == 1:
        break
else:
    print('Else block!')
```

```
>>>
Loop 0
Loop 1
```

还有一个奇怪的地方：如果 for 循环要遍历的序列是空的，那么就会立刻执行 else 块。

```
for x in []:
    print('Never runs')
else:
    print('For Else block!')
```

>>>
For Else block!

初始循环条件为 false 的 while 循环，如果后面跟着 else 块，那它也会立刻执行。

```
while False:
    print('Never runs')
else:
    print('While Else block!')
```

>>>
While Else block!

知道了循环后面的 else 块所表现出的行为之后，我们会发现：在搜索某个事物的时候，这种写法是有意义的。例如，要判断两个数是否互质（也就是判断两者除了 1 之外，是否没有其他的公约数），可以把有可能成为公约数的每个值都遍历一轮，逐个判断两数是否能以该值为公约数。尝试完每一种可能的值之后，循环就结束了。如果两个数确实互质，那么在执行循环的过程中，程序就不会因 break 语句而跳出，于是，执行完循环后，程序会紧接着执行 else 块。

```
a = 4
b = 9
for i in range(2, min(a, b) + 1):
    print('Testing', i)
    if a % i == 0 and b % i == 0:
        print('Not coprime')
        break
else:
    print('Coprime')
```

>>>
Testing 2
Testing 3
Testing 4
Coprime

实际上，我们不会这样写代码，而是会用辅助函数来完成计算。这样的辅助函数，有两种常见的写法。

第一种写法是，只要发现被测参数符合自己想要搜寻的条件，就尽早返回。如果整

有关此电子书的说明

本人可以帮助你找到你要的PDF电子书，计算机类，文学，艺术，设计，医学，理学，经济，金融等等。质量都很清晰，为方便读者阅读观看，每本100%都带可跳转的书签索引和目录，只要您提供给我书的相关信息，一般我都能找到，如果您有需求，请联系我 QQ1779903665。

PDF代找说明：

本人已经帮助了上万人找到了他们需要的PDF，其实网上有很多PDF,大家如果在网上不到的话，可以联系我QQ，大部分我都可以找到，而且每本100%带书签索引目录。因PDF电子书都有版权，请不要随意传播，如果您有经济购买能力，请尽量购买正版。

提供各种书籍的pd电子版代找服务，如果你找不到自己想要的书的pdf电子版，我们可以帮您找到，如有需要，请联系 QQ 1779903665.

备用:QQ 461573687

声明：本人只提供代找服务，每本100%索引书签和目录，因寻找和后期制作pdf电子书有一定难度，仅收取代找费用。如因PDF产生的版权纠纷，与本人无关，我们仅仅只是帮助你寻找到你要的pdf而已。